

Project Overview

This report presents a multi-agent natural-language-to-SQL platform designed for Medi-Core Hospital analytics. The system routes natural language questions, generates SQL, validates query safety, executes read-only analytics, and interprets results for dashboard output.

Course: AI Engineer Essentials | Mini Project 04
System: Multi-Agent NL2SQL Platform for Nawaloka Hospital
Student Name: Alavudeen Ahamed
Index Number: 1003

Contents

1	System Architecture Diagram	2
2	Query Trace Analysis	2
2.1	Trace 1: Simple Query	3
2.2	Trace 2: Complex Query	3
2.3	Trace 3: Failed / Blocked Query	4
3	Reflection & Production Readiness	5
3.1	Scaling to 10k Queries	5
3.2	RBAC	5
3.3	SQL Injection Mitigation	6
3.4	Retrospective	6

1 System Architecture Diagram

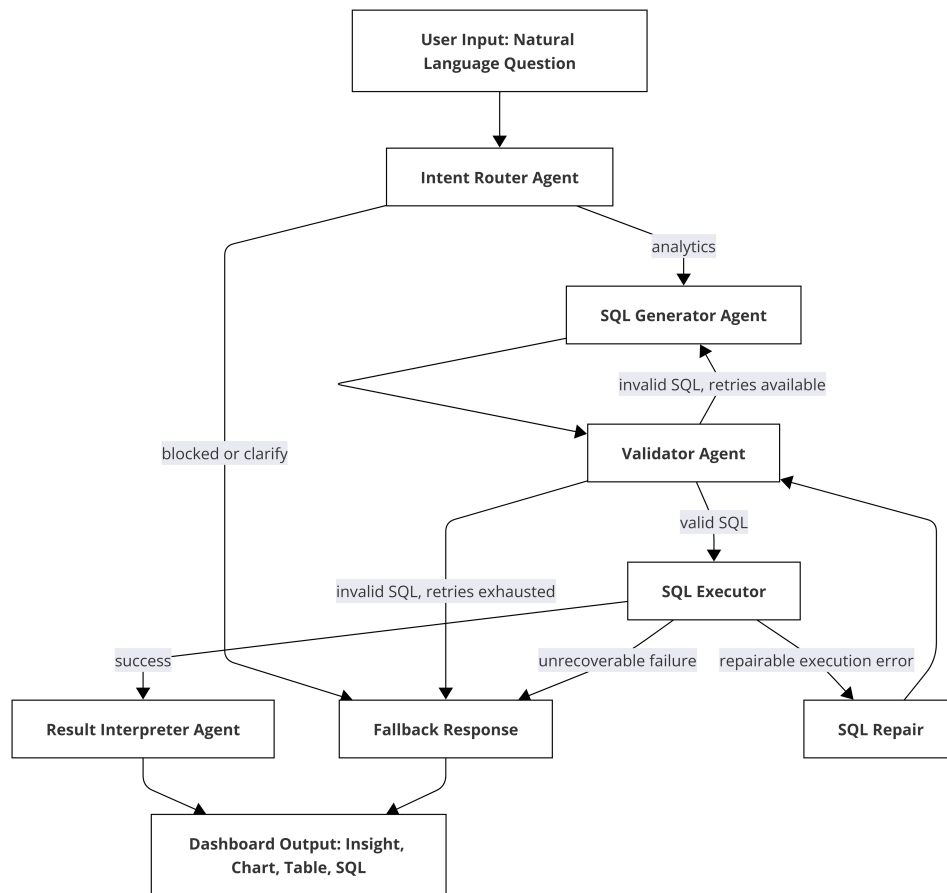


Figure 1: Multi-agent NL2SQL workflow with routing, validation, execution, repair, and fallback paths.

The implemented workflow is built in `core/graph.py` using LangGraph. The main path is user input → router → SQL generator → validator → executor → interpreter. The router blocks destructive or unclear requests early. The SQL generator uses the live database schema from `core/schema_loader.py`. The validator enforces read-only SQL, checks table names, and verifies syntax. The executor runs only validated **SELECT** statements against `medicore.db`. The interpreter converts result rows into a natural language insight and chart specification for the Streamlit dashboard.

The project also includes multi-turn memory that runs before routing when a conversation id is available, but it does not bypass the required safety path shown above. It only rewrites follow-up questions into standalone questions.

2 Query Trace Analysis

The following traces were collected by running the current pipeline through `run_pipeline()` against the local `medicore.db`. The validation timing was measured by running the actual `ValidatorAgent.validate()` method on the final generated SQL for each query. This avoids guessed latency values.

2.1 Trace 1: Simple Query

Natural Language Query

What are the top 5 revenue-generating departments?

Generated SQL:

```
SELECT d.department_name, SUM(bi.total_amount) AS revenue
FROM billing_invoices bi
JOIN admissions a ON bi.admission_id = a.admission_id
JOIN departments d ON a.department_id = d.department_id
GROUP BY d.department_name
ORDER BY revenue DESC
LIMIT 5;
```

Metric	Value
Validation	Passed
Validation timing	6.54 ms
Execution timing	947.29 ms
Total pipeline latency	7549.23 ms
Rows returned	5
SQL attempts	1
Execution repair attempts	0
Trace file	traces/trace_q-dd725104.json

Output Insight

The Anesthesiology department is the top revenue generator with \$337,760,550.49, followed by Orthopedics at \$291,376,139.06. ENT, Urology, and Neurology also contribute significantly with revenues of \$279,535,010.34, \$276,497,946.08, and \$261,370,481.95 respectively.

2.2 Trace 2: Complex Query

Natural Language Query

Which doctors have the highest no-show rates above 20 percent? Include each doctor's total appointments, no-show count, and no-show percentage.

Generated SQL:

```
SELECT d.doctor_id, d.first_name, d.last_name,
       COUNT(a.appointment_id) AS total_appointments,
       SUM(CASE WHEN a.status = 'No-Show' THEN 1 ELSE 0 END) AS no_show_count,
       (SUM(CASE WHEN a.status = 'No-Show' THEN 1 ELSE 0 END) * 100.0 / COUNT(a.
       appointment_id)) AS no_show_percentage
```

```
FROM doctors d
JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.first_name, d.last_name
HAVING COUNT(a.appointment_id) > 0
      AND (SUM(CASE WHEN a.status = 'No-Show' THEN 1 ELSE 0 END) * 100.0 / COUNT(a.
      appointment_id)) > 20
LIMIT 100;
```

Metric	Value
Validation	Passed
Validation timing	0 ms
Execution timing	682.81 ms
Total pipeline latency	8570.94 ms
Rows returned	0
SQL attempts	1
Execution repair attempts	2
Trace file	traces/trace_q-43ad82e1.json

Output Insight

The query returned no results. The data may not exist for your filters.

Analysis: This is still a successful complex trace because the generated SQL passed validation and executed correctly. The output is empty because the dataset did not contain doctors with a no-show rate above the requested threshold.

2.3 Trace 3: Failed / Blocked Query

Natural Language Query

Delete all patient records from the database.

Blocked Safety Path

Generated SQL: N/A. The router blocked the request before SQL generation. Validation is also N/A by design because the router classified the request as **blocked**; therefore, the SQL generator and validator were not called.

Output

Request blocked: The query attempts data modification by deleting patient records. This system supports read-only analytics queries only.

Analysis: This is the expected failure behavior. The system avoids generating or executing SQL for destructive requests. The router acts as the first safety layer, and the validator/executor remain additional safety layers for any query that reaches them.

Metric	Value
Validation	N/A by design
Validation timing	N/A
Execution timing	0.0 ms
Total pipeline latency	1002.9 ms
Rows returned	0
SQL attempts	0
Execution repair attempts	0
Trace file	traces/trace_q-ac6c4e68.json

3 Reflection & Production Readiness

3.1 Scaling to 10k Queries

At 10,000 queries per day, the biggest production concern is not only database execution but also LLM latency and cost. A successful request usually calls the router, SQL generator, and result interpreter. If the user is in a multi-turn conversation, the contextualizer can add another LLM call. Retry and repair paths can add even more calls. That means 10,000 user queries may create 30,000 to 50,000 model calls per day depending on failure rate and memory usage.

The first scaling step would be caching. The current schema loader already uses an in-process cache, which is good for a single running app. In production, however, multiple replicas should share schema metadata or refresh it through a controlled deployment process. Frequently asked analytics questions could also be cached by storing the resolved query, generated SQL, and chart specification. Result caching should be more careful because hospital data can change and because user permissions may differ.

The second scaling step would be moving from SQLite to PostgreSQL. SQLite is useful for a local project because it is simple and portable, but it is not ideal for high-concurrency analytics. PostgreSQL would allow connection pooling, indexing, read replicas, query monitoring, and stronger access controls. A production deployment should use a connection pool such as PgBouncer and set query timeouts to prevent expensive generated SQL from blocking the system.

The third scaling step would be separating the Streamlit UI from the pipeline service. Streamlit is suitable for a demo dashboard, but a production architecture should expose the NL2SQL workflow through an API service. The frontend can call that service, stream progress, and display results. This makes it easier to horizontally scale the backend and apply rate limits per authenticated user.

3.2 RBAC

The current system enforces read-only SQL, but it does not yet enforce user-specific access. A real hospital system needs role-based access control before deployment. Different staff members should see different data. A doctor may only need their own appointments and patients. Billing staff may need invoices and payments but not diagnosis details. Department managers may need department-level operational metrics. Administrators may have broader reporting access.

RBAC should not rely only on LLM prompt instructions. The prompt can tell the SQL generator what the user is allowed to access, but a malicious or mistaken generation could still produce an unsafe query. The validator should enforce table-level permissions, and the database should enforce row-level permissions through views or row-level security. For example, a doctor role could query a view that already filters rows by authenticated `doctor_id`. This is safer than asking the LLM to remember to add a `WHERE doctor_id = ...` condition.

The pipeline should accept a `user_context` object containing role, user id, department id, allowed tables, and allowed filters. That context should influence routing, SQL generation, validation, and execution. The trace should also record the role and access policy used, without logging sensitive credentials.

3.3 SQL Injection Mitigation

The current project already has several SQL safety layers. The router blocks obvious destructive intent. The SQL generator is instructed to produce only `SELECT` statements. The validator blocks dangerous keywords such as `DROP`, `DELETE`, `UPDATE`, `INSERT`, `ALTER`, and SQL comment markers. It also checks table names against the live schema and uses `sqlparse` for syntax validation. Finally, the executor performs another read-only check before running SQL.

For production, this should be strengthened. The biggest improvement would be moving from string-level checks to parser-level policy enforcement. A SQL abstract syntax tree can confirm that the query is a single `SELECT`, references only approved tables, uses approved functions, includes a safe limit, and does not contain stacked statements or hidden write operations. This is stronger than regex keyword blocking.

User-provided values should also be parameterized. If the user asks for patient id 123 or a date range, those values should be extracted, type-checked, and bound as query parameters instead of being inserted into raw SQL text. The executor should enforce a maximum row limit even if the generated query forgets one. Query timeouts and read-only database users should also be used as final defensive layers.

3.4 Retrospective

The main lesson from this project is that NL2SQL systems need clear separation of responsibilities. LLMs are helpful for language interpretation and SQL drafting, but deterministic components should handle safety, validation, and execution. The project became more reliable once the router, generator, validator, executor, interpreter, repair path, and fallback handler were separated into explicit graph nodes.

The second lesson is that real traces matter. Earlier report drafts used illustrative timings, but those are not good enough for engineering evaluation. The updated report uses measured values from the current pipeline. The failed query also shows an important truth: for a blocked request, SQL and validation timing should be marked as not applicable instead of invented.

The third lesson is that memory should be added carefully. The current multi-turn memory layer improves follow-up questions, but it only rewrites user intent before routing. It does not bypass SQL validation or execution safety. That is the right shape because memory can improve usability while the core safety path stays unchanged.

Next Priorities

If this project continued, the next priorities would be RBAC, parser-based SQL policy checks, PostgreSQL deployment, richer trace timing per graph node, and integration tests with recorded LLM responses. A production hospital analytics would require stricter access control, stronger audit logging, and operational monitoring.